Product Requirements Document — Lab Sample Tracking & KDS Dashboard

Working name: LabBoard (placeholder) **Owner:** Shubham **Context:** Environmental testing laboratory (water, air, soil, effluent, etc.) **Status:** Draft v1 — for review before build **Last updated:** 6 June 2026

---

1. Summary

A web app that tracks every sample through the lab from the moment it is received until the final report is dispatched, and surfaces live status on a kitchen-display-style board (KDS). Each sample is registered, assigned to one of ~10 categories (which determines its test panel), worked on by analysts who tick off each test, then passed through technical review and authorised approval before the report goes out. The board is the centrepiece: a wall-mountable, auto-refreshing view of where every sample sits and which ones are running late.

This document treats categories and their test panels as configurable master data, not hardcoded values, so the lab can edit them without a code change.

---

2. Problem statement

Sample status today lives in people's heads, on paper registers, or in spreadsheets. Nobody can answer "what's pending, who's holding it, and what's overdue" without asking around. There is no single screen the whole lab can look at. Bottlenecks (a sample stuck at review for two days) are invisible until a client chases the report.

---

3. Goals and non-goals

Goals

- Single source of truth for sample status across the whole lab.
- A live board, glanceable from across the room, that shows every sample's current stage.
- Make overdue / at-risk samples obvious via turnaround-time (TAT) colour coding.
- Capture who did what and when (audit trail) for accountability and NABL-style traceability.
- Reduce "where is sample X?" interruptions to zero.

Non-goals (for v1)

- Not a full LIMS. No instrument integration, no calibration management, no inventory/reagent tracking.
- Not generating the final NABL report PDF in v1 (status tracking only; report generation is a later phase — see §12).
- No billing or invoicing.
- No client-facing portal in v1.

---

4. Users and roles

Role	Does	Sees
Front desk / receiver	Registers incoming samples, generates sample ID	Inward form, today's intake
Lab manager	Assigns category + test panel, allocates tests to analysts	Full board, assignment screen
Analyst	Performs assigned tests, enters result, ticks done	Personal work queue + board
Reviewer (QC)	Technical review of completed results; pass or send back	Review queue + board
Authorised signatory	Final sign-off; approve or reject	Approval queue + board
Admin	Manages users, categories, test panels, settings	Everything + master data

A single person may hold several roles (common in a small lab). Roles are permission flags, not exclusive accounts.

---

5. Domain model (key concepts)

- **Sample** — a single physical item received for testing. Has one category, one client/source, and a set of tests. Moves through the lifecycle in §6.
- **Category** — one of ~10 sample types (e.g. drinking water, effluent, ambient air, stack emission, soil, noise, sludge/hazardous waste, microbiology, food, others). Each category owns a default **test panel**.
- **Test (parameter)** — a single measurement on a sample (e.g. pH, BOD, COD, TSS, PM10, SO_2). Belongs to a category's panel. Has a status of its own (pending → in progress → done)

and a result value.

- **Test panel** — the list of tests that come pre-attached when a sample is given a category. Editable per sample (add/remove individual tests).
- **Analyst assignment** — a test (or set of tests) allocated to a specific analyst.
- **Lifecycle status** — the sample's current stage (§6).
- **TAT (turnaround time)** — the agreed number of days/hours to complete a sample; drives the colour coding.

> The exact 10 categories and the parameter list per category are placeholders above — confirm the real lab list and seed them as master data.

---

6. Sample lifecycle (state machine)

States, in order, with allowed transitions:

#	Status	Set by	Next
1	`registered`	Front desk	→ assigned
2	`assigned`	Lab manager	→ in_analysis
3	`in_analysis`	Analyst (starts work)	→ review (when all tests done)
4	`under_review`	auto (all tests ticked)	→ approved **or** → in_analysis (rework)
5	`approved`	Signatory	→ reported **or** → under_review (rework)
6	`reported`	auto / front desk	→ closed
7	`closed`	auto	terminal

Rework loops (the important bit for the board):

- Reviewer **rejects** at step 4 → sample drops back to `in_analysis`, flagged with a rework reason; affected tests reopen.
- Signatory **rejects** at step 5 → sample drops back to `under_review` with a reason.

Every transition writes an audit record (who, when, from-status, to-status, optional note).

A sample can only enter `under_review` when **every** test in its panel is marked done. This rule is enforced server-side.

---

7. Functional requirements (by module)

7.1 Sample inward / registration

- Form to register a new sample: client/source name, sampling location, sampling date/time, received date/time, sample type, **category**, quantity, preservation/condition, received-by, remarks.
- Auto-generate a unique, human-readable **sample ID**, e.g. `SGE/<category-code>/<YYYY>/<seq>` (`SGE/WW/2026/0042`). Sequence resets per year; format configurable.
- On save: status = `registered`; the category's default test panel is attached automatically.
- Optional: bulk intake (register N samples in one batch with shared client/date).

7.2 Category & test assignment

- Lab manager opens a `registered` sample, confirms/edits the test panel (add or remove individual tests).
- Allocate each test (or the whole panel) to an analyst.
- On assignment: status → `assigned`; each test → `pending`.
- TAT/due date set here (default from category, editable).

7.3 Analyst workbench

- Each analyst has a personal queue of tests assigned to them, grouped by sample, sorted by due date (most urgent first).
- For each test: start (→ `in progress`), enter result value + unit, mark done (the **tick**). Optional method/instrument note.
- Progress indicator per sample: "6 / 10 tests done".
- When the last test of a sample is ticked, sample auto-moves to `under_review`.

7.4 QC / technical review

- Reviewer queue shows samples in `under_review`.
- Reviewer sees all results side by side, can approve (→ `approved`) or send back (→ `in_analysis`) with a mandatory reason; can reopen specific tests.

7.5 Final approval

- Signatory queue shows `approved`-pending samples.
- Approve (→ `reported`/ready) or reject (→ `under_review`) with reason. Records signatory identity + timestamp.

7.6 KDS dashboard — see §8 (the core deliverable).

7.7 Admin / master data

- CRUD for users + roles.
- CRUD for categories (name, code, default TAT, default test panel).
- CRUD for tests/parameters (name, unit, default method, which categories).
- Settings: sample-ID format, TAT thresholds for colour coding, working hours.

7.8 Audit & history

- Every status change and every test action logged with user + timestamp.
- A per-sample timeline view ("registered 10:04 by Asha → assigned 11:20 by Manager → …").

---

8. KDS dashboard specification (the core)

A board, like a kitchen display, designed to live on a wall-mounted screen and update itself.

Layout

- **Columns = lifecycle stages**: Registered · Assigned · In analysis · Under review · Approved · Reported. (Closed hidden by default.)
- **Cards = samples.** Each card flows left-to-right through the columns as its status changes.
- Column headers show a live count ("In analysis · 7").

Card contents

- Sample ID (large, primary).
- Category (with a small colour dot per category).
- Client/source (secondary).
- Test progress bar / "6 / 10".
- Assigned analyst initials/avatar.
- Time-in-stage and a TAT indicator.
- A rework badge if the sample was sent back.

TAT colour coding (glanceable urgency)

- **Green —** comfortably within TAT.
- **Amber —** approaching due (e.g. >70% of TAT elapsed).
- **Red —** overdue, or sent back for rework.
- Thresholds configurable in settings.

Behaviour

- **Real-time**: cards move and counts update without a manual refresh (Supabase Realtime subscription on the samples table). Fallback poll every N seconds.

- **Filters**: by category, by analyst, by overdue-only, by date range. Filters persist per device.
- **Sort within a column**: most-overdue first by default.
- **Click a card** → sample detail (full info, test list with statuses, timeline).
- **TV/kiosk mode**: full-screen, larger text, no chrome, auto-rotate filters optional. Read-only.
- **Empty/booted state** handled gracefully (no flash of empty board on load).

Secondary views (same data, different lens)

- **My queue** (analyst-centric list).
- **Overdue list** (everything red/amber, across all stages).
- A small KPI strip: registered today, reported today, average TAT, samples overdue.

---

9. Non-functional requirements

- **Responsive**: works on a wall TV, a desktop, and a phone (analysts on the floor).
- **Performance**: board renders <1.5s with up to ~500 active samples; real-time update latency <2s.
- **Auth & access**: Supabase Auth; row-level security so analysts see their queue, managers/board see all, master data is admin-only.
- **Reliability**: optimistic UI on the tick action with rollback on failure.
- **Auditability**: immutable audit log (append-only).
- **Offline tolerance** (nice-to-have): the tick action queues if the network blips.

---

10. Data model (first cut)

```
users            (id, name, email, role_flags, active)
categories       (id, code, name, default_tat_hours, color, active)
tests            (id, name, unit, default_method, active)
category_tests   (category_id, test_id)              -- the default panel
samples          (id, sample_code, category_id, client, source,
                  sampling_at, received_at, received_by, quantity,
                  condition, remarks, status, due_at, created_at)
sample_tests     (id, sample_id, test_id, assigned_to, status,
                  result_value, result_unit, method, done_at)
status_events    (id, sample_id, from_status, to_status, by_user,
                  note, created_at)                  -- audit / timeline
```

`samples.status` is the enum from §6. `sample_tests.status` is `pending | in_progress | done`. The "all tests done → under_review" rule and the rework reopen are enforced in a server action / Postgres function, not the client.

11. Tech stack (recommended)

Matches your existing NexHire setup so it's familiar:

- **Frontend**: Next.js 15 (App Router) + TypeScript + Tailwind CSS + Framer Motion (card movement animations).
- **Backend/DB**: Supabase — Postgres + Auth + Row-Level Security + **Realtime** (this is what makes the board live).
- **Hosting**: Vercel.
- **Optional AI later**: Groq (e.g. a "summarise today's bottlenecks" assistant) — not needed for v1.

Card-movement animations and the real-time subscription are the two things worth prototyping first, since they carry the whole experience.

---

12. Scope and phasing

MVP (v1) — get the board working end to end

1. Auth + roles + RLS.
2. Master data: categories, tests, panels (seeded).
3. Sample inward (single registration) + auto sample ID.
4. Category confirm + test assignment to analysts.
5. Analyst workbench: enter result + tick.
6. Auto-advance to review; review pass/reject.
7. Approval pass/reject.
8. **KDS board** with columns, cards, TAT colours, real-time, click-through detail, TV mode.
9. Audit timeline.

Phase 2

- Bulk intake, overdue list + KPI strip, my-queue refinements, filters persistence.
- Report PDF generation (NABL-style template) + dispatch tracking.

Phase 3

- Client portal (status lookup by sample ID).
- AI bottleneck summariser, analytics dashboard, exports.

---

13. Open questions / assumptions to confirm

1. **The real 10 categories** and each one's actual parameter list — the §5 list is a placeholder.
2. Is the test count always ~10, or does it vary by category? (Design assumes variable.)
3. Sample ID format — confirm `SGE/<code>/<year>/<seq>` or your existing register convention.
4. Are review and approval always two separate people, or can one person do both?
5. TAT — is it per category, per client, or per test? (Design assumes per category, editable per sample.)
6. Do results need numeric limits / pass-fail against a standard in v1, or just recorded values?
7. NABL traceability requirements — anything beyond who/when that must be captured for v1?

Once §13 is settled, this PRD is ready to drive a Claude Code build. The natural first prompt is the Supabase schema + RLS, then the KDS board against seeded data.